

Question 1

Correct

Marked out of 1.00

Flag question

Given below is a simple program written in C language.

Change the text in the code given below to make the program print "Hello C" instead of "Hello B".

Answer: (penalty regime: 0 %)

Reset answer

```
1 #include <stdio.h>
2
3 int main()
4 {
5     printf("Hello C");
6     return 0;
7 }
```

	Expected	Got	
✓	Hello C	Hello C	✓

Passed all tests! ✓

Question 2

Correct

Marked out of 1.00

Flag question

The code given below contains instructions to print the text "I love Apples" to the console.

The \n in the text "I love Apples\n" ensures that the line breaks after printing the text "I love Apples" (which means that nothing else is printed on the same line).

Follow the steps given below to change the text, execute **compile** command and finally **execute** the file :

1. In the code given below, change the text to print "I love Mangoes" instead of "I love Apples".

Answer: (penalty regime: 0 %)

Reset answer

```
1 #include <stdio.h>
2
3 int main()
4 {
5     printf("I love Mangoes\n");
6     return 0;
7 }
```

	Expected	Got	
✓	I love Mangoes	I love Mangoes	✓

Passed all tests! ✓

```
int main()
{
    // this is an end of line comment
    printf("I love C Language!");
    return 0;
}
```

Answer: (penalty regime: 0 %)

```
1 1 /*
2   This is a sample C program
3   developed by REC
4   */
5   #include <stdio.h>
6
7   int main()
8   {
9       //this is an end of line comment
10      printf("I love C Language!");
11      return 0;
12  }
13 }
```

	Expected	Got	
✓	I love C Language!	I love C Language!	✓

Passed all tests! ✓

Question **2**

Correct

Marked out of 1.00

Flag question

The code given below contains text that prints "**DennisRitchieBrianKernighan**".

Make the suggested changes to the code so that it prints "**DennisRitchieBrianKernighan**" as shown below.

Dennis Ritchie
Brian Kernighan

To make the required changes, follow the steps given below to introduce the SPACE character and the \n new line character appropriately:

1. Insert a space between "Dennis" and "Ritchie". Make sure that no extra space or any other character apart from space are inserted.
2. Insert a \n between "Ritchie" and "Brian" Make sure that no extra space or any other character apart from \n are inserted.
3. Insert a space between "Brian" and "Kernighan". Make sure that no extra space or any other character apart from space are inserted.

Answer: (penalty regime: 0 %)

Reset answer

```
1 #include <stdio.h>
2
3 int main()
4 {
5     printf("Dennis Ritchie\nBrian Kernighan");
6     return 0;
7 }
```

	Expected	Got	
✓	Dennis Ritchie Brian Kernighan	Dennis Ritchie Brian Kernighan	✓

Passed all tests! ✓

Read the code given below and retype in the space provided. **Note** the effects of \t and \n in the resulting output when executed successfully.

Content to be reproduced

```
#include <stdio.h>
```

```
int main()
{
    printf("One Two");
    printf("Three\n");
    printf("Four\nFive\n");
    return 0;
}
```

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2
3 int main()
4 {
5     printf("One Two");
6     printf("Three\n");
7     printf("Four\nFive\n");
8     return 0;
9 }
```

	Expected	Got	
✓	One TwoThree Four Five	One TwoThree Four Five	✓

Passed all tests! ✓

Question **4**

Correct

Marked out of
1.00

Flag question

Make the following changes in the code given below:

1. Comment the statement which prints **"Mango"**.
2. Remove the comment on the statement which prints **"Banana"**.

Answer: (penalty regime: 0 %)

Reset answer

```
1 #include <stdio.h>
2
3 int main()
4 {
5     printf("Orange\n");
6     //printf("Mango\n");
7     printf("Banana");
8     return 0;
9 }
```

	Expected	Got	
✓	Orange Banana	Orange Banana	✓

Passed all tests! ✓

Question 1

Correct

Marked out of
1.00

Flag question

Read the code given below to learn naming conventions in identifiers.

For example, consider the program given below:

```
#include <stdio.h>

int main()
{
    int age = 2; // age is an integer variable

    int firstNumber = 2; // firstNumber is an integer variable

    // If there are two or more words in an identifier/variable - User can also use "camel

    int second_number = 3; // second_number is an integer variable

    // Any space cannot be used between two words of an identifier/variable; User can use u

    int _i_am_also_a_valid_identifier = 4; // _i_am_also_a_valid_identifier is an integer v

    // An identifier/variable name must be start with an alphabet or underscore (_) only, n

    printf("age = %d\n", age);
    printf("firstNumber = %d\n", firstNumber);
    printf("second_number = %d\n", second_number);
    printf("_i_am_also_a_valid_identifier = %d\n", _i_am_also_a_valid_identifier);
    return 0;
}
```

Fill in the missing code in the below program to print the values of the given variables.

Answer: (penalty regime: 0 %)

Reset answer

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int age = 2;
6     int firstNumber = 2;
7     int second_number = 3;
8     int _i_am_also_a_valid_identifier = 4;
9     printf("age = %d\n", age); // Fill in the missing code
10    printf("firstNumber = %d\n",firstNumber); // Fill in the missing code
11    printf("second_number = %d\n",second_number); // Fill in the missing code
12    printf("_i_am_also_a_valid_identifier = %d\n",_i_am_also_a_valid_identi
13    return 0;
14 }
```

	Expected	Got	
✓	age = 2 firstNumber = 2 second_number = 3 _i_am_also_a_valid_identifier = 4	age = 2 firstNumber = 2 second_number = 3 _i_am_also_a_valid_identifier = 4	✓

Passed all tests! ✓

Question 1

Correct

Marked out of
1.00

Flag question

Click on **Check** without correcting the code.

This results in many errors because the main function is not defined correctly.

Now, correct the spelling of the main function and submit the program once again.

Answer: (penalty regime: 0 %)

Reset answer

```
1 #include <stdio.h>
2
3 int main()
4 {
5     printf("Correct Me!");
6     return 0;
7 }
```

	Expected	Got	
✓	Correct Me!	Correct Me!	✓

Passed all tests! ✓

Question 2

Correct

Marked out of
1.00

Flag question

Identify and correct the error in the code given below.

Answer: (penalty regime: 0 %)

Reset answer

```
1 #include <stdio.h>
2
3 int main()
4 {
5     printf("Hello, float data type allocates 4 bytes in memory");
6     return 0;
7 }
```

	Expected	Got
✓	Hello, float data type allocates 4 bytes in memory	Hello, float data type allocates 4 by

Passed all tests! ✓

Question 3

Correct

Marked out of
1.00

Flag question

Identify and correct the error in the code given below.

Answer: (penalty regime: 0 %)

Reset answer

```
1 #include <stdio.h>
2
3 int main()
4 {
5     printf("Hello, # is a preprocessor in C");
6     return 0;
7 }
```

	Expected	Got	
✓	Hello, # is a preprocessor in C	Hello, # is a preprocessor in C	✓

Passed all tests! ✓

Question 4

Correct

Marked out of
1.00

Flag question

Identify and correct the error in the code given below.

Answer: (penalty regime: 0 %)

Reset answer

```
1 #include <stdio.h>
2 int main()
3 {
4     printf("Hello, I am learning C Language!");
5     return 0;
6 }
```

	Expected	Got	
✓	Hello, I am learning C Language!	Hello, I am learning C Language!	✓

Passed all tests! ✓

Question 5

Correct

Marked out of
1.00

Flag question

In **C** programming language, execution of the code starts with a **function** called **main**.

We shall learn more about functions in the later sections. For now, we can safely assume that **function** is the name given to a set of one or more executable statements. **main()** is a **user defined function**, i.e., a user (a programmer) writes the code for the **main()** function.

While executing a **C** program, the **Operating System (OS)** only calls the **main()** function in that program.

When the **OS** executes a program, the program usually returns an integer value 0 if the execution of that program is successful.

In **C**, **main()** can be written in such a way that it returns a an int.

```
#include <stdio.h>
```

```
int main()
{
    printf("Sample main() function with int as return type!");
    return 0; // 0 value indicates that the execution is successful
}
```

If the programmer does not specify any return type, the return type is by default considered as int.

The name of the **main()** function should always be in lowercase, i.e., if a function is written as **Main()**, it is not the main function which is called by the **OS**.

Read the code given below to familiarize yourself with the syntax of **main()** function. Retype in the space provided.

```
#include <stdio.h>
```

```
int main()
{
    printf("Impossible is nothing!");
    return 0;
}
```

Answer: (penalty regime: 0 %)

```
1  #include <stdio.h>
2
3  int main()
4  {
5      printf("Impossible is nothing!");
6      return 0;
7  }
```

	Expected	Got	
✓	Impossible is nothing!	Impossible is nothing!	✓

Passed all tests! ✓

Question 1

Correct

Marked out of
1.00

Flag question

Identify the error and correct the code. [Hint: Verify if all variables are declared before they are first used.]

Answer: (penalty regime: 0 %)

Reset answer

```

1 #include <stdio.h>
2
3 int main()
4 {
5     int number1 = 20, number2 = 30;
6     int sub = number1 - number2;
7     printf("The difference of the two given numbers = %d\n", sub);
8     return 0;
9 }
10

```

	Expected	Got
✓	The difference of the two given numbers = -10	The difference of the two given numbers =

Passed all tests! ✓

Question 2

Correct

Marked out of
1.00

Flag question

In the program given below, we shall learn how to assign values to int data type from binary, octal, hex and character literals.

Read the code given below and retype in the space provided.

```

#include <stdio.h>

int main()
{
    int binaryThree = 0b11;
    printf("binaryThree value = %d\n", binaryThree);
    int octalEight = 010;
    printf("octalEight value = %d\n", octalEight);
    int hexTen = 0xA;
    printf("hexTen value = %d\n", hexTen);
    int asciiValueOfOne = '1';
    printf("asciiValueOfOne value = %d\n", asciiValueOfOne);
    int asciiValueOfA = 'A';
    printf("asciiValueOfA value = %d\n", asciiValueOfA);
    return 0;
}

```

Answer: (penalty regime: 0 %)

```

1 #include <stdio.h>
2 int main()
3 {
4     int binaryThree = 0b11;
5     printf("binaryThree value = %d\n", binaryThree);
6     int octalEight = 010;
7     printf("octalEight value = %d\n", octalEight);
8     int hexTen = 0xA;
9     printf("hexTen value = %d\n", hexTen);
10    int asciiValueOfOne = '1';
11    printf("asciiValueOfOne value = %d\n", asciiValueOfOne);
12    int asciiValueOfA = 'A';
13    printf("asciiValueOfA value = %d\n", asciiValueOfA);
14    return 0;
15 }

```

	Expected	Got	
✓	binaryThree value = 3 octalEight value = 8 hexTen value = 10 asciiValueOfOne value = 49	binaryThree value = 3 octalEight value = 8 hexTen value = 10 asciiValueOfOne value = 49	✓

Question 3

Correct

Marked out of 1.00

Flag question

In the program given below, fill in the missing code to add two integer numbers.

Answer: (penalty regime: 0 %)

Reset answer

```

1 #include <stdio.h>
2
3 int main()
4 {
5     int num1 = 15, num2 = 25, sum;
6     printf("Given integers are num1 = %d, num2 = %d\n", num1, num2);
7     sum = num1+num2; //Write the code to add num1 and num2 and place the result in sum
8     printf("Sum of 2 given numbers = %d\n", sum);
9     return 0;
10 }
```

	Expected	Got	
✓	Given integers are num1 = 15, num2 = 25 Sum of 2 given numbers = 40	Given integers are num1 = 15, num2 = 25 Sum of 2 given numbers = 40	✓

Passed all tests! ✓

Question 4

Correct

Marked out of 1.00

Flag question

To print unsigned values on the console, use %u format character instead of %d in the **printf()** function.

Whenever an attempt is made to assign a negative number to an **unsigned int** (For eg: unsigned int num = -1;) the compiler does not flag it as an **error**. Instead, it will automatically convert the negative number to a positive number as shown below:

```

unsigned int num = -1;
The value stored in num = unsigned int maximum_value + 1 - num;
The final value in num = 4294967295 (in a 32-bit processing system)
```

In the program given below, fill in the missing **format characters** to print **signed** and **unsigned** values.

Answer: (penalty regime: 0 %)

Reset answer

```

1 #include <stdio.h>
2
3 int main()
4 {
5     signed int number1 = -20, number2 = 20;
6     unsigned int number3 = -1, number4 = 1;
7     printf("Given signed values are %d and %d\n", number1, number2); // Fill in the missing format characters
8     printf("Given unsigned values are %u and %u\n", number3, number4); // Fill in the missing format characters
9     return 0;
10 }
```

	Expected	Got	
✓	Given signed values are -20 and 20 Given unsigned values are 4294967295 and 1	Given signed values are -20 and 20 Given unsigned values are 4294967295 and 1	✓

Passed all tests! ✓

Question 5

Correct

Marked out of 1.00

Flag question

To print unsigned values on the console, use %u format character instead of %d in the **printf()** function.

Whenever an attempt is made to assign a negative number to an **unsigned int** (For eg: unsigned int num = -1;) the compiler does not flag it as an **error**. Instead, it will automatically convert the negative number to a positive number as shown below:

```
unsigned int num = -1;
The value stored in num = unsigned int maximum_value + 1 - num;
The final value in num = 4294967295 (in a 32-bit processing system)
```

In the program given below, fill in the missing **format characters** to print **signed** and **unsigned** values.

Answer: (penalty regime: 0 %)

Reset answer

```
1 #include <stdio.h>
2
3 int main()
4 {
5     signed int number1 = -20, number2 = 20;
6     unsigned int number3 = -1, number4 = 1;
7     printf("Given signed values are %d and %d\n", number1, number2); // Fill
8     printf("Given unsigned values are %u and %d\n", number3, number4); // Fill
9     return 0;
10 }
```

	Expected	Got	
✓	Given signed values are -20 and 20 Given unsigned values are 4294967295 and 1	Given signed values are -20 and 20 Given unsigned values are 4294967295 and 1	✓
Passed all tests! ✓			

Question 6

Correct

Marked out of 1.00

Flag question

To print unsigned values on the console, use %u format character instead of %d in the **printf()** function.

Whenever an attempt is made to assign a negative number to an **unsigned int** (For eg: unsigned int num = -1;) the compiler does not flag it as an **error**. Instead, it will automatically convert the negative number to a positive number as shown below:

```
unsigned int num = -1;
The value stored in num = unsigned int maximum_value + 1 - num;
The final value in num = 4294967295 (in a 32-bit processing system)
```

In the program given below, fill in the missing **format characters** to print **signed** and **unsigned** values.

Answer: (penalty regime: 0 %)

Reset answer

```
1 #include <stdio.h>
2
3 int main()
4 {
5     signed int number1 = -20, number2 = 20;
6     unsigned int number3 = -1, number4 = 1;
7     printf("Given signed values are %d and %d\n", number1, number2); // Fill
8     printf("Given unsigned values are %u and %d\n", number3, number4); // Fill
9     return 0;
10 }
```

	Expected	Got	
✓	Given signed values are -20 and 20 Given unsigned values are 4294967295 and 1	Given signed values are -20 and 20 Given unsigned values are 4294967295 and 1	✓
Passed all tests! ✓			

Question 7

Correct

Marked out of
1.00

Flag question

To print unsigned values on the console, use %u format character instead of %d in the **printf()** function.

Whenever an attempt is made to assign a negative number to an **unsigned int** (For eg: unsigned int num = -1;) the compiler does not flag it as an **error**. Instead, it will automatically convert the negative number to a positive number as shown below:

```
unsigned int num = -1;
```

The value stored in num = unsigned int maximum_value + 1 - num;

The final value in num = 4294967295 (in a 32-bit processing system)

In the program given below, fill in the missing **format characters** to print **signed** and **unsigned** values.

Answer: (penalty regime: 0 %)

Reset answer

```
1 #include <stdio.h>
2
3 int main()
4 {
5     signed int number1 = -20, number2 = 20;
6     unsigned int number3 = -1, number4 = 1;
7     printf("Given signed values are %d and %d\n", number1, number2); // Fill
8     printf("Given unsigned values are %u and %d\n", number3, number4); // Fill
9     return 0;
10 }
```

	Expected	Got	
✓	Given signed values are -20 and 20 Given unsigned values are 4294967295 and 1	Given signed values are -20 and 20 Given unsigned values are 4294967295 and 1	✓

Passed all tests! ✓

Question 1

Correct

Marked out of 1.00

Flag question

Identify and correct the errors in the code given below:

Answer: (penalty regime: 0 %)

Reset answer

```
1 #include <stdio.h>
2
3 int main()
4 {
5     float num1 = 5.345f, num2 = 12.4, result;
6     printf("Given float values are num1 = %f, num2 = %f\n", num1, num2);
7     result = num1 / num2;
8     printf("Result of division = %f\n", result);
9     return 0;
10 }
```

	Expected	Got
✓	Given float values are num1 = 5.345000, num2 = 12.400000 Result of division = 0.431048	Given float values are num1 = 5 Result of division = 0.431048

Passed all tests! ✓

Question 2

Correct

Marked out of 1.00

Flag question

Identify and correct the errors in the code given below:

Expected Output:

Given float values are num1 = 5.340000, num2 = 125.789001

The result after dividing in float format = 23.555992

The result after dividing in exponential format = 2.355599e+01

Answer: (penalty regime: 0 %)

Reset answer

```
1 #include <stdio.h>
2
3 int main()
4 {
5     float num1 = 5.34, num2 = 125.789f, result;
6     printf("Given float values are num1 = %f, num2 = %f\n", num1, num2);
7     result = num2 / num1;
8     printf("The result after dividing in float format = %f\n", result);
9     printf("The result after dividing in exponential format = %e\n", result);
10    return 0;
11 }
```

	Expected	Got
✓	Given float values are num1 = 5.340000, num2 = 125.789001 The result after dividing in float format = 23.555992 The result after dividing in exponential format = 2.355599e+01	Given float values are nu The result after dividing The result after dividing

Passed all tests! ✓

Question 1

Correct

Marked out of
1.00

Flag question

Arithmetic operators can be used in expressions with operands of different data types.

For example, the **addition operator** (+) can be used on **int** and **float** data types with the result being a **float** data type.

If the same is performed on **float** and **double** data types, the result will be a **double** data type, i.e., the result will always be of the **data type** with the highest memory capacity.

Consider the following examples with different combinations of arithmetic operators and data types.

Expression	Result	Description
2 + 2.5	4.500000	int + float = float
'A' + 10	75	char + int = int
5.9 + 4.67L	10.570000	float + long double = long double

Fill in the code given below with the correct missing **format characters**:

Answer: (penalty regime: 0 %)

Reset answer

```

1 #include <stdio.h>
2
3 int main()
4 {
5     printf("Result1 = %f\n", (2 + 2.5));
6     printf("Result2 = %d\n", ('A' + 10));
7     printf("Result3 = %Lf\n", (5.9 + 4.67L));
8     return 0;
9 }
```

	Expected	Got	
✓	Result1 = 4.500000 Result2 = 75 Result3 = 10.570000	Result1 = 4.500000 Result2 = 75 Result3 = 10.570000	✓

Passed all tests! ✓

Question 2

Correct

Marked out of
1.00

Flag question

The table given below demonstrates the use of various **arithmetic operators** using two variables c1 and c2 of type char with values 'A' and 'D' respectively:

Expression Result

c1 65

c1 + c2 133

c1 + c2 + 5 138

c1 + c2 + '5' 186

In the above examples, the character 'A' is substituted with its ASCII value **65** and 'D' is substituted with 68. The character '5' is substituted with its ASCII value 53. The integer value 5 is used as it is.

The following table demonstrates the usage of various **arithmetic operators** using two variables a and b of type int with values 11 and -3 respectively:

Expression Result

a + b 8

a - b 14

a * b -33

a / b -3

a % b 2

In the program given below, type the missing code to find the **result** of applying different **arithmetic operators** on **char** data type values.

Answer: (penalty regime: 0 %)

Reset answer

```

1  #include <stdio.h>
2
3  int main()
4  {
5      char c1 = 'A', c2 = 'D';
6      printf("c1 = %d\n", c1);
7      printf("c1 + c2 = %d\n", (c1 + c2));
8      printf("c1 + c2 + 5 = %d\n", (c1 + c2 + 5));
9      printf("Result = %d", (c1 + c2 + '5'));
10     return 0;
11 }
```

	Expected	Got	
✓	c1 = 65	c1 = 65	✓
	c1 + c2 = 133	c1 + c2 = 133	
	c1 + c2 + 5 = 138	c1 + c2 + 5 = 138	
	Result = 186	Result = 186	

Passed all tests! ✓

Question 3

Correct

Marked out of
1.00

Flag question

Division of one integer by another integer is referred to as **integer** division. This operation always results in an integer with truncated quotient.

If a **division** operation is carried out with two **floating point numbers** or with one **floating point number** and one **integer**, the result will be a **floating point quotient**.

The table given below demonstrates the usage of various **arithmetic operators** using two variables num1 and num2 of type float with values 12.5 and 2.0 respectively:

Expression	Result
num1 + num2	14.500000
num1 - num2	10.500000
num1 * num2	25.000000
num1 / num2	6.250000

Note that the **remainder operator** (%) is not applicable for **floating point numbers**.

In the program given below, type the missing code to find the **result** of applying different **arithmetic operators** on **floating point numbers**.

Answer: (penalty regime: 0 %)

Reset answer

```

1 #include <stdio.h>
2
3 int main()
4 {
5     float num1 = 12.5, num2 = 2.0;
6     printf("Result of addition = %f\n", (num1 + num2));
7     printf("Result of subtraction = %f\n", (num1 - num2));
8     printf("Result of multiplication = %f\n", (num1 * num2));
9     printf("Result of division = %f\n", (num1 / num2));
10    return 0;
11 }
```

	Expected	Got	
✓	Result of addition = 14.500000 Result of subtraction = 10.500000 Result of multiplication = 25.000000 Result of division = 6.250000	Result of addition = 14.500000 Result of subtraction = 10.500000 Result of multiplication = 25.000000 Result of division = 6.250000	✓

Passed all tests! ✓

The format for usage of arithmetic operator is as follows:

operand1operatoroperand2

According to the coding conventions in C, a single space should be provided to the left and to the right of an operator.

The table given below demonstrates the use of various **arithmetic operators** using two variables num1 and num2 of type int with values 10 and 3 respectively:

Expression	Result
num1 + num2	13
num1 - num2	7
num1 * num2	30
num1 / num2	3
num1 % num2	1

Read the code given below to understand the usage of **arithmetic operators**. Retype in the space provided.

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int num1 = 10, num2 = 3;
```

```
    printf("Addition Result = %d\n", (num1 + num2));
```

```
    printf("Subtraction Result = %d\n", (num1 - num2));
```

```
    printf("Multiplication Result = %d\n", (num1 * num2));
```

```
    printf("Division Result = %d\n", (num1 / num2));
```

```
    printf("Remainder = %d", (num1 % num2));
```

```
    return 0;
```

```
}
```

Answer: (penalty regime: 0 %)

```
1  #include <stdio.h>
2  int main()
3  {
4      int num1 = 10, num2 = 3;
5      printf("Addition Result = %d\n", (num1 + num2));
6      printf("Subtraction Result = %d\n", (num1 - num2));
7      printf("Multiplication Result = %d\n", (num1*num2));
8      printf("Division Result = %d\n", (num1/num2));
9      printf("Remainder = %d", (num1%num2));
10 }
```

	Expected	Got	
✓	Addition Result = 13 Subtraction Result = 7 Multiplication Result = 30 Division Result = 3 Remainder = 1	Addition Result = 13 Subtraction Result = 7 Multiplication Result = 30 Division Result = 3 Remainder = 1	✓

Passed all tests! ✓

operator.

The table given below demonstrates the use of various **relational and equality operators** using variables `int num1 = 7;`, `float num2 = 5.5;`, `char ch = 'w';`

Expression	Interpretation	Result Value
<code>(num1 > 5)</code>	true	1
<code>((num1 + num2) <= 10)</code>	false	0
<code>(ch == 119)</code>	true	1
<code>(ch != 'p')</code>	true	1
<code>(ch >= 10 * (num1 + num2))</code>	false	0

Read the code given below and retype in the space provided.

```
#include <stdio.h>
```

```
int main()
{
    int num1 = 7;
    float num2 = 5.5;
    char ch = 'w';
    printf("Result1 = %d\n", (num1 > 5));
    printf("Result2 = %d\n", ((num1 + num2) <= 10));
    printf("Result3 = %d\n", (ch == 119));
    printf("Result4 = %d\n", (ch != 'p'));
    printf("Result5 = %d", (ch >= 10 * (num1 + num2)));
    return 0;
}
```

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2 int main()
3 {
4     int num1 = 7;
5     float num2 = 5.5;
6     char ch = 'w';
7     printf("Result1 = %d\n", (num1 > 5));
8     printf("Result2 = %d\n", ((num1 + num2) <= 10));
9     printf("Result3 = %d\n", (ch == 119));
10    printf("Result4 = %d\n", (ch != 'p'));
11    printf("Result5 = %d\n", (ch >= 10*(num1 + num2)));
12    return 0;
13
14 }
```

	Expected	Got	
✓	Result1 = 1	Result1 = 1	✓
	Result2 = 0	Result2 = 0	
	Result3 = 1	Result3 = 1	
	Result4 = 1	Result4 = 1	
	Result5 = 0	Result5 = 0	

Passed all tests! ✓

According to the coding conventions in C, a single space should be provided to the left and to the right of the operator.

The below table demonstrates the use of various **relational and equality operators** using variables `int num1 = 7;`, `float num2 = 5.5;`, `char ch = 'w';`:

Expression	Interpretation	Result Value
<code>(num1 >= 6) && (ch == 'w')</code>	true	1
<code>(num2 < 11) && (num1 > 100)</code>	false	0
<code>(ch != 'p') ((num1 + num2) <= 10)</code>	true	1
<code>!(num1 > (num2 + 1))</code>	false	0
<code>!(num1 <= 3)</code>	true	1

Read the code given below and retype in the space provided.

```
#include <stdio.h>
```

```
int main()
{
    int num1 = 7;
    float num2 = 5.5;
    char ch = 'w';
    printf("Result1 = %d\n", ((num1 >= 6) && (ch == 'w')));
    printf("Result2 = %d\n", ((num2 < 11) && (num1 > 100)));
    printf("Result3 = %d\n", ((ch != 'p') || ((num1 + num2) <= 10)));
    printf("Result4 = %d\n", !(num1 > (num2 + 1)));
    printf("Result5 = %d\n", !(num1 <= 3));
    return 0;
}
```

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int num1 = 7;
6     float num2 = 5.5;
7     char ch = 'w';
8     printf("Result1 = %d\n", ((num1 >= 6) && (ch == 'w')));
9     printf("Result2 = %d\n", ((num2 < 11) && (num1 > 100)));
10    printf("Result3 = %d\n", ((ch != 'p') || ((num1 + num2) <= 10)));
11    printf("Result4 = %d\n", !(num1 > (num2 + 1)));
12    printf("Result5 = %d\n", !(num1 <= 3));
13    return 0;
14 }
```

	Expected	Got	
✓	Result1 = 1	Result1 = 1	✓
	Result2 = 0	Result2 = 0	
	Result3 = 1	Result3 = 1	
	Result4 = 0	Result4 = 0	
	Result5 = 1	Result5 = 1	

Passed all tests! ✓

Question 1

Correct

Marked out of
1.00

Flag question

Read the code given below to understand the working of unary operators. Retype in the space provided.

```
#include <stdio.h>

int main()
{
    int x = 16;
    printf("+x = %d\n", (+x));
    printf("-x = %d\n", (-x));
    printf("x = %d\n", x);
    printf("++x = %d\n", (++x));
    printf("x = %d\n", x);
    printf("x++ = %d\n", (x++));
    printf("x = %d\n", x);
    printf("--x = %d\n", (--x));
    printf("x = %d\n", x);
    printf("x-- = %d\n", (x--));
    printf("x = %d", x);
    return 0;
}
```

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2 int main()
3 {
4     int x = 16;
5     printf("+x = %d\n", (+x));
6     printf("-x = %d\n", (-x));
7     printf("x = %d\n", x);
8     printf("++x = %d\n", (++x));
9     printf("x = %d\n", x);
10    printf("x++ = %d\n", (x++));
11    printf("x = %d\n", x);
12    printf("--x = %d\n", (--x));
13    printf("x = %d\n", x);
14    printf("x-- = %d\n", (x--));
15    printf("x = %d\n", x);
16    return 0;
17 }
```

	Expected	Got	
✓	+x = 16	+x = 16	✓
	-x = -16	-x = -16	
	x = 16	x = 16	
	++x = 17	++x = 17	
	x = 17	x = 17	
	x++ = 17	x++ = 17	
	x = 18	x = 18	
	--x = 17	--x = 17	
	x = 17	x = 17	
	x-- = 17	x-- = 17	
	x = 16	x = 16	

Passed all tests! ✓

Question 2

Correct

Marked out of
1.00

Flag question

Read the code given below to understand the working of **increment** and **decrement** operators. Retype in the space provided.

```
#include <stdio.h>
```

```
int main()
{
    int x = 4, y;
    y = x++;
    printf("y = %d x = %d\n", y, x);
    y = ++x;
    printf("y = %d x = %d\n", y, x);
    y = x--;
    printf("y = %d x = %d\n", y, x);
    y = --x;
    printf("y = %d x = %d\n", y, x);
    return 0;
}
```

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2 int main()
3 {
4     int x = 4, y;
5     y = x++;
6     printf("y = %d x = %d\n", y, x);
7     y = ++x;
8     printf("y = %d x = %d\n", y, x);
9     y = x--;
10    printf("y = %d x = %d\n", y, x);
11    y = --x;
12    printf("y = %d x = %d\n", y, x);
13    return 0;
14 }
```

	Expected	Got	
✓	y = 4 x = 5	y = 4 x = 5	✓
	y = 6 x = 6	y = 6 x = 6	
	y = 6 x = 5	y = 6 x = 5	
	y = 4 x = 4	y = 4 x = 4	

Passed all tests! ✓

Name SHRINITHI V 2025-BME

Question 1

Correct

Marked out of
1.00

Flag question

Read the code given below to understand the usage of the assignment operator. Retype in the space provided.

```
#include <stdio.h>
```

```
int main()
{
    int x = 24, y = 39, z = 45;
    z = x + y;
    y = z - y;
    x = z - y;
    printf("x = %d y = %d z = %d", x, y, z);
    return 0;
}
```

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2 int main()
3 {
4     int x = 24, y = 39, z = 45;
5     z = x+y;
6     y = z-y;
7     x = z-y;
8     printf("x = %d y = %d z = %d" ,x,y,z);
9     return 0;
10 }
```

	Expected	Got	
✓	x = 39 y = 24 z = 63	x = 39 y = 24 z = 63	✓

Passed all tests! ✓

Question 2

Correct

Marked out of
1.00

Flag question

Read the code given below and retype in the space provided.

```
#include <stdio.h>

int main()
{
    int x = 2, y = 18, z = 12;
    x += y;
    printf("x = %d\n", x);
    y *= 2;
    printf("y = %d\n", y);
    z /= 5;
    printf("z = %d\n", z);
    x %= 7;
    printf("x = %d", x);
    return 0;
}
```

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2 int main()
3 {
4     int x =2, y=18, z=12;
5     x+=y;
6     printf("x = %d\n",x);
7     y*= 2;
8     printf("y = %d\n",y);
9     z/=5;
10    printf("z = %d\n",z);
11    x%=7;
12    printf("x = %d",x);
13    return 0;
14 }
```

	Expected	Got	
✓	x = 20	x = 20	✓
	y = 36	y = 36	
	z = 2	z = 2	
	x = 6	x = 6	

Passed all tests! ✓

```
#include <stdio.h>
```

```
int main()
{
    int marks = 75, pass_marks = 50;
    (marks > pass_marks) ? printf("Passed C exam.") : printf("Failed C exam.");
    return 0;
}
```

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2 int main()
3 {
4     int marks = 75, pass_marks = 50;
5     (marks > pass_marks) ? printf("Passed C exam.") : printf("Failed C exam.");
6     return 0;
7 }
```

	Expected	Got	
✓	Passed C exam.	Passed C exam.	✓

Passed all tests! ✓

Question 2

Correct

Marked out of 1.00

Flag question

In the program given below, fill in the missing code to find the **largest** of the two given numbers using **ternary operator**.

Answer: (penalty regime: 0 %)

Reset answer

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int num1 = 20, num2 = 25, large;
6     large = (num1 > num2) ? num1 : num2; // Write the correct code
7     printf("Largest number = %d", large);
8     return 0;
9 }
```

	Expected	Got	
✓	Largest number = 25	Largest number = 25	✓

Passed all tests! ✓

Question 1

Correct

Marked out of
1.00

Flag question

There was a large ground in center of the city which is rectangular in shape. The Corporation decides to build a Cricket stadium in the area for school and college students, But the area was used as a car parking zone. In order to protect the land from using as an unauthorized parking zone, the corporation wanted to protect the stadium by building a fence. In order to help the workers to build a fence, they planned to place a thick rope around the ground. They wanted to buy only the exact length of the rope that is needed. They also wanted to cover the entire ground with a carpet during rainy season. They wanted to buy only the exact quantity of carpet that is needed. They requested your help. Can you please help them by writing a program to find the exact length of the rope and the exact quantity of carpet that is required?

Input format:

Input consists of 2 integers. The first integer corresponds to the length of the ground and the second integer corresponds to the breadth of the ground.

Output Format:

Output Consists of two integers. The first integer corresponds to the length. The second integer corresponds to the quantity of carpet required.

Sample Input:

50

20

Sample Output:

140

1000

For example:

Input	Result
50	140
20	1000

Answer: (penalty regime: 0 %)

```

1  #include <stdio.h>
2  int main()
3  {
4      int length, breadth;
5      int rope, carpet;
6      scanf("%d", &length);
7      scanf("%d", &breadth);
8      rope = 2*(length + breadth);
9      carpet = length * breadth;
10     printf("%d\n", rope);
11     printf("%d", carpet);
12     return 0;
13 }
```

	Input	Expected	Got	
✓	50	140	140	✓
	20	1000	1000	

Passed all tests! ✓

Question 1

Correct

Marked out of
1.00

Flag question

Training for sports day has begun and the physical education teacher has decided to conduct some team games. The teacher wants to split the students in higher secondary into equal sized teams. In some cases, there may be some students who are left out from the teams and he wanted to use the left out students to assist him in conducting the team games. For instance, if there are 50 students in a class and if the class has to be divided into 7 equal sized teams, 7 students will be there in each team and 1 student will be left out. That 1 student will assist the PET. With this idea in mind, the PET wants your help to automate this team splitting task. Can you please help him out?

INPUT FORMAT:

Input consists of 2 integers. The first integer corresponds to the number of students in the class and the second integer corresponds to the number of teams.

OUTPUT FORMAT:

The output consists of two integers. The first integer corresponds to the number of students in each team and the second integer corresponds to the students who are left out.

SAMPLE INPUT:

60

8

SAMPLE OUTPUT:

7

4

For example:

Input	Result
60	7
8	4

Answer: (penalty regime: 0 %)

```

1 #include <stdio.h>
2 int main()
3 {
4     int students, teams;
5     int each_team, left_out;
6     scanf("%d", &students);
7     scanf("%d", &teams);
8     each_team = students / teams;
9     left_out = students % teams;
10    printf("%d\n", each_team);
11    printf("%d", left_out);
12    return 0;
13 }
```

	Input	Expected	Got	
✓	60	7	7	✓
	8	4	4	

Passed all tests! ✓

Question 1

Correct

Marked out of 1.00

Flag question

Each Sunday, a newspaper agency sells w copies of a special edition newspaper for Rs. x per copy. The cost to the agency of each newspaper is Rs. y . The agency pays a fixed cost for storage, delivery and so on of Rs.100 per Sunday. The newspaper agency wants to calculate the profit which it obtains only on Sundays. Can you please help them out by writing a program to compute the profit if w , x , and y are given?

INPUT FORMAT:

Input consists of 3 integers: w , x , and y . w is the number of copies sold, x is the cost per copy and y is the cost the agency spends per copy.

OUTPUT FORMAT:

The output consists of a single integer which corresponds to the profit obtained by the newspaper agency.

SAMPLE INPUT:

1000

2

1

SAMPLE OUTPUT:

900

For example:

Input	Result
1000	900
2	
1	

Answer: (penalty regime: 0 %)

```

1 #include <stdio.h>
2 int main()
3 {
4     int w, x, y;
5     int profit;
6     scanf("%d", &w);
7     scanf("%d", &x);
8     scanf("%d", &y);
9     profit = (w*x) - (w*y) - 100;
10    printf("%d", profit);
11    return 0;
12 }
```

	Input	Expected	Got	
✓	1000	900	900	✓
	2			
	1			

Passed all tests! ✓

Question 1

Correct

Marked out of
1.00

Flag question

Four kids Peter, Susan, Edmond and Lucy travel through a wardrobe to the land of Narnia. Narnia is a fantasy world of magic with mythical beasts and talking animals. While exploring the land of narnia Lucy found Mr. Tumnus the two legged stag ,and she followed it, down a narrow path .She and Mr. Tumnus became friends and he offered a cup of coffee to Lucy in his small hut. It was time for Lucy to return to her family and so she bid good bye to Mr. Tumnus and while leaving Mr. Tumnus told that it is quite difficult to find the route back as it was already dark. He told her to see the trees while returning back and said that the first tree with two digits number will help her find the way and the way to go back to her home is the sum of digits of the tree and that numbered way will lead her to the tree next to the wardrobe where she can find the others. Lucy was already confused, so please help her in finding the route to her home....

Input Format:

Input consists of an integer corresponding to the 2-digit number.

Output Format:

Output consists of an integer corresponding to the sum of its digits.

SAMPLE INPUT:

87

SAMPLE OUTPUT:

15

For example:

Input	Result
87	15

Answer: (penalty regime: 0 %)

```

1 #include <stdio.h>
2 int main()
3 {
4     int num, digit1, digit2, sum;
5     scanf("%d", &num);
6     digit1 = num / 10;
7     digit2 = num % 10;
8     sum = digit1 + digit2;
9     printf("%d", sum);
10    return 0;
11 }
```

	Input	Expected	Got	
✓	87	15	15	✓
✓	54	9	9	✓

Passed all tests! ✓